CTE 2059 - Computer Graphic
Lecture 1 – Scene & Primitives

University of Engineering and Technology
VIETNAM NATIONAL UNIVERSITY, HANOI

Outline

- **Getting Started**
  - Installation
  - Creating a Scene
- **Fundamentals**
- **Primitives**
  - Box, Sphere, Cylinder, Torus, Cone, Capsule, Plane
  - Grouping & Hierarchy
- **Hands-On Activity**

Getting Started

Installation

- **Installing VSCode:** https://code.visualstudio.com/
- **Installing Node.js:** https://nodejs.org/en/download (Window Installer)
- Open VSCode, create a new folder, enable AutoSave option and open a Terminal
- **Check installation: Type these commands in the terminal**
  - node --version => You should see a version number
  - npm --version => You should see a version number
- **Installing Three.js: Type these commands in the terminal**
  - npm install --save three
  - npm install --save-dev vite
- **Installing VSCode Live Server Extension** (Optional)

How to run the code?

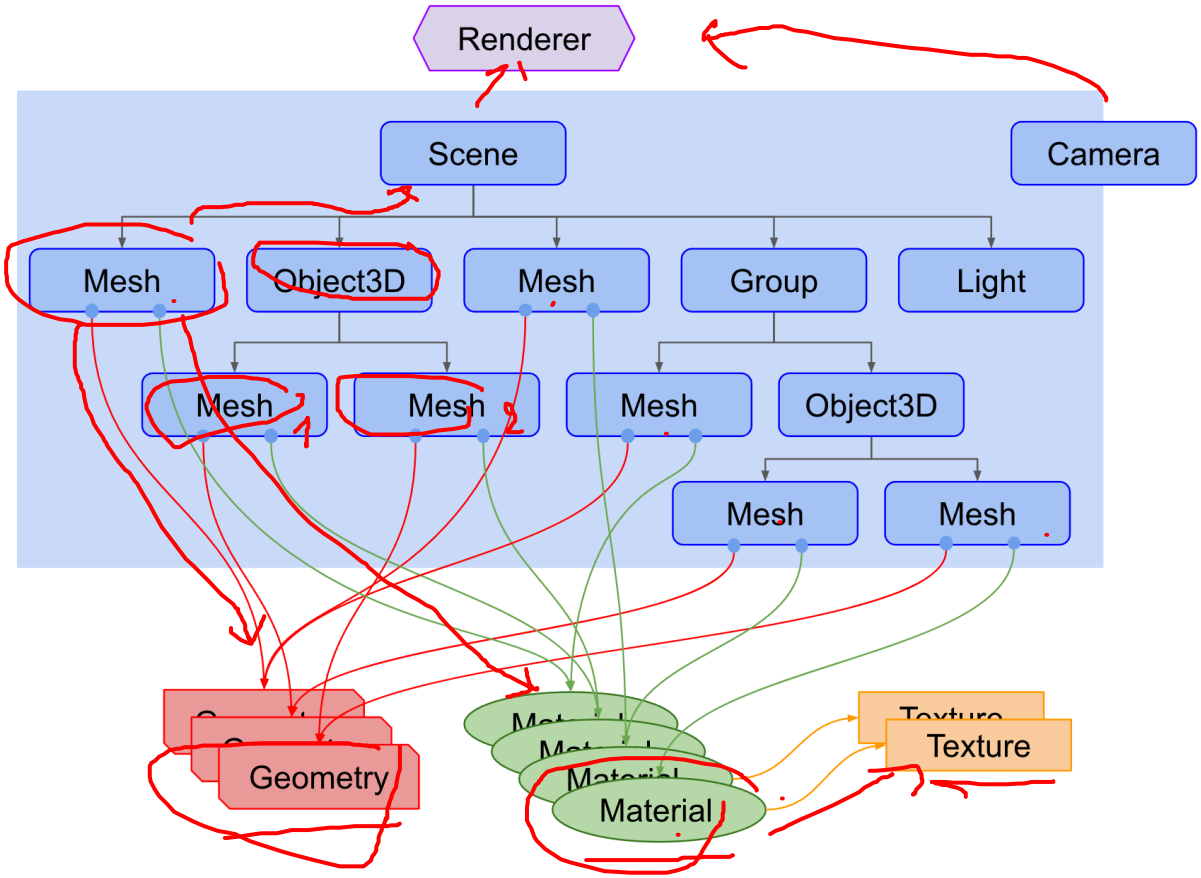
- **Basic Structure:**
  - At least one HTML file to define the webpage.
  - At least a JavaScript file to run your three.js code.
  - Static folder: resources such as textures, audio, 3D models will go here.
- **How to run the code:** (using source code from three.js manual)
  - From your terminal, run: **npx vite**
  - From your html file, "Go Live" with Live Server Extension.
  - You should see a localhost URL linked to your web application

    (It should be a page with a green cube in the middle and a dark background at this point).

Fundamentals

Structure of a Three.js app

- Here is a diagram that represents a **small three.js app:**



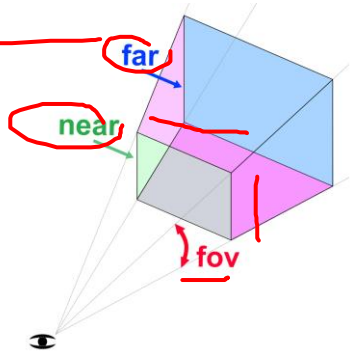
“Hello Cube” Setup

Creating the scene:

```
import * as THREE from 'three';

const scene = new THREE.Scene();
const camera = new THREE.PerspectiveCamera( 75, window.innerWidth /
window.innerHeight, 0.1, 1000 );

const renderer = new THREE.WebGLRenderer();
renderer.setSize( window.innerWidth, window.innerHeight );
document.body.appendChild( renderer.domElement );
```

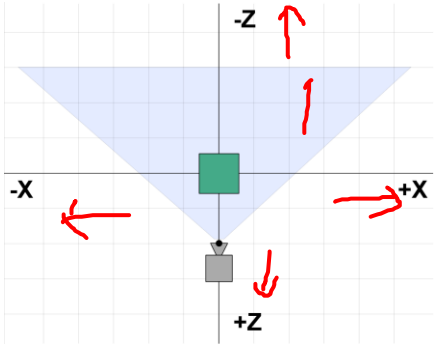
Rendering:

```
function animate( time ) {
  renderer.render( scene, camera );
}
renderer.setAnimationLoop( animate );
```



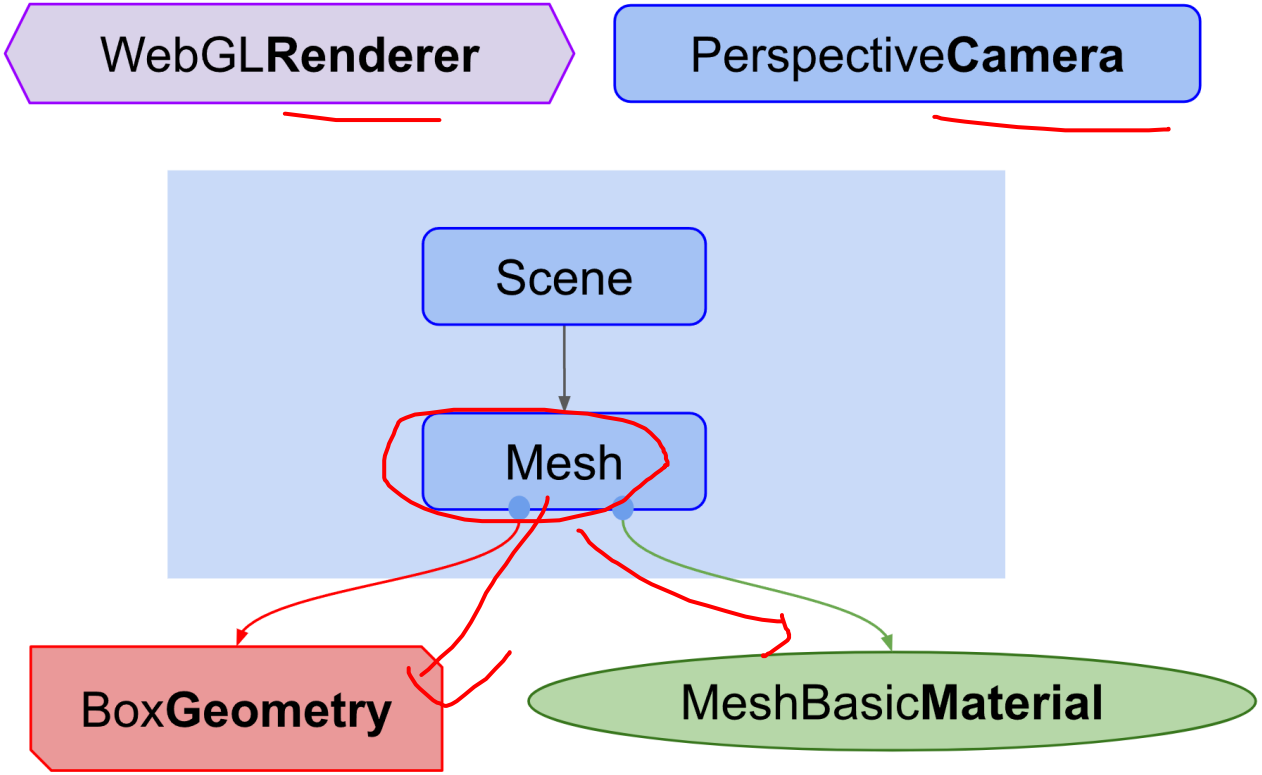
Adding a cube:

```
const geometry = new THREE.BoxGeometry( 1, 1, 1 );
const material = new THREE.MeshBasicMaterial( { color: 0x00ff00 } );
const cube = new THREE.Mesh( geometry, material );
scene.add( cube );

camera.position.z = 5;
```



Structure of a Three.js app

- The **"Hello Cube"** setup looks like this:



Primitives

Creating a 3D object

Let's take a look at how the cube was created!

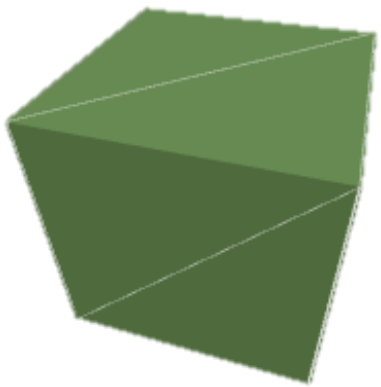
```
const geometry = new THREE.BoxGeometry( 1, 1, 1 );
const material = new THREE.MeshBasicMaterial( { color: 0x00ff00 } );
const cube = new THREE.Mesh( geometry, material );
scene.add( cube );

camera.position.z = 5;
```

To create a basic 3D object, we need a mesh. In three.js a mesh represents the combination of three things:

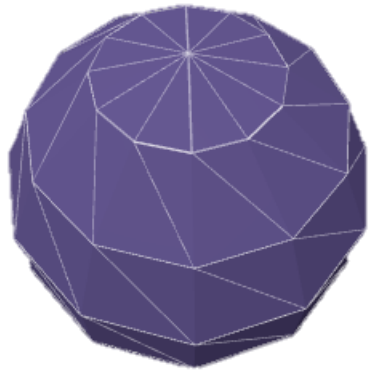
1. A geometry (the shape of the object)
2. A material (how to draw the object, what color, texture, etc.)
3. The position, orientation, and scale of that object in the scene relative to its parent.

Note: In the previous example, that parent is the scene.

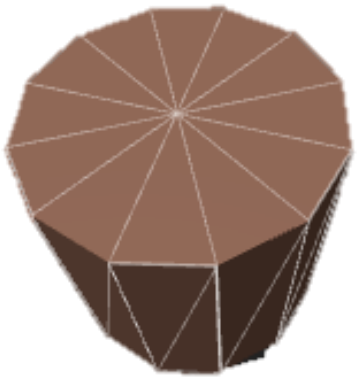
Some commonly used Primitives



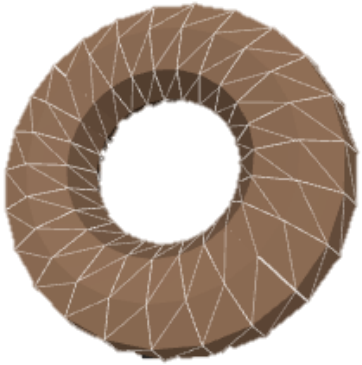
Box Geometry



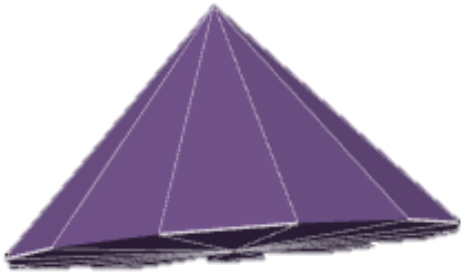
Sphere Geometry



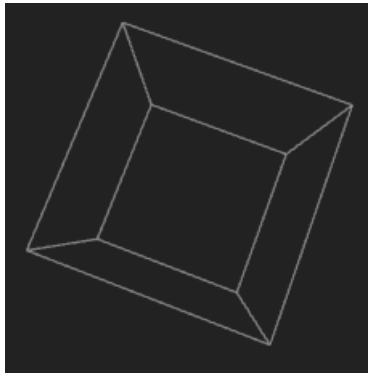
Cylinder Geometry



Torus Geometry



Cone Geometry



Can you guess what is this?

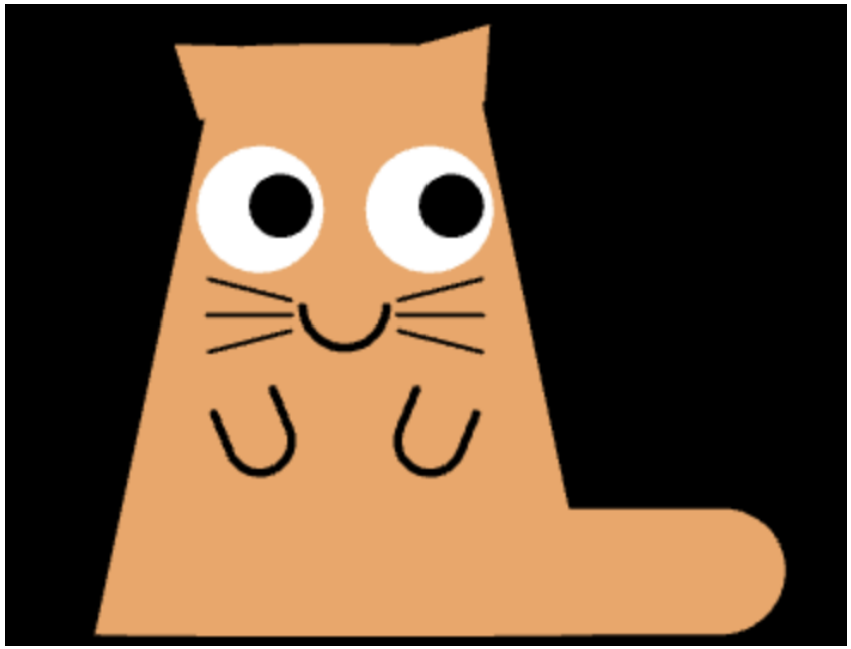
Grouping & Hierarchy

This is almost identical to an **Object3D**.
Its purpose is to make working with groups of objects syntactically clearer.

```
// Create a group and add the two cubes.
// These cubes can now be rotated/scaled, etc., as a group.
const group = new THREE.Group();
group.add( meshA );
group.add( meshB );
scene.add( group );
```

Hands-On Activity
How to draw a cat?

Hands-On Activity



Activity: How to draw a cat?

In this activity, you are provided with a sample picture of an orange cat.

Your tasks are:

- Identify the structural characteristics of the cat (i.e., which shapes form the cat's structure).
- Implementing these shapes in Three.js
- Create a function to group all these shapes and return a single object that represents the cat.